Язык программирования Python

Перегрузка операций

Доцент кафедры ИВТ, к.т.н.
Проскурин Александр Викторович

Класс

Класс – это пользовательский тип данных, в котором объединены переменные других типов **данных** (**поля**) и **функции**, обрабатывающие эти переменные (**методы**).

Как и любой тип, класс отвечает за две вещи:

1) какие значения можно хранить в переменных (**экземплярах / объектах**) этого класса;
2) какие действия над переменными этого класса можно совершать.

Допустимые действия, как правило, задаются пользовательскими методами.

Программист сам присваивает им удобные имена, задаёт список параметров и т.д.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Перегрузка операций

Помимо написания обычных методов, в языке Python есть возможность **перегружать различные операции языка с помощью специальных методов**.

Под операциями в данном контексте понимаются не только знаки сложения (+), умножения (*), проверки на равенство (==) и т.п., но также особенности синтаксиса языка, обеспечивающие операции создания объекта, вызова объекта как функции, обращение к элементу поля объекта по индексу, преобразование объекта в строку и другое.

Таким образом, перегруженные операции позволяют классу определять свое поведение, когда его объект используется в качестве операнда в выражениях унарных или бинарных операций.

Они также служат реализациями, вызываемыми некоторыми встроенными функциями (такими как len(), str() и т.д.).

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Ограничения на перегрузку операций

При перегрузке операций нужно учитывать следующие особенности:

- **Имена методов**, используемых для перегрузки операций, встроены в язык Python, их нельзя изменить и **нужно запомнить** (все они имеют двойное подчеркивание в начале и конце).
- Нельзя определить новую операцию, например, ++.
- Количество операндов (соответственно, количество параметров), приоритет выполнения и ассоциативность операций (слева направо или справа налево) не изменяется.
- Как минимум один операнд должен быть пользовательского типа. Реализовать перегрузку операций для встроенных типов **нельзя**.

Примечание. Перегружайте операции только тогда, когда смысл перегрузки очевиден и не несёт скрытых сюрпризов! Например, операция + очевидна для матриц, но неочевидна для ящиков.

Перегрузка операций является одним из способов реализации принципа **полиморфизма**.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Перегрузка операций сравнения

Операции сравнения сравнивают значения **двух операндов**, не изменяя их, и возвращают **результат типа bool**. Для перегрузки операций используются следующие методы:

Специальный метод	Операция	Значение
`__lt__(self, other)`	`x < y`	Возвращает True, если x меньше, чем y
`__le__(self, other)`	`x <= y`	Возвращает True, если x меньше или равно y
`__eq__(self, other)`	`x == y`	Возвращает True, если x равно y
`__ne__(self, other)`	`x != y`	Возвращает True, если x не равно y
`__gt__(self, other)`	`x > y`	Возвращает True, если x больше, чем y
`__ge__(self, other)`	`x >= y`	Возвращает True, если x больше или равно y

При перегрузке операций сравнения достаточно прописать действия только для двух (например, == и <), после чего выразить оставшиеся через них.

Пример: x > y равно y < x; x >= y равно y < x and y == x и т.д.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример перегрузки операций сравнения (1)

```
class Time: # Класс моделирует временной промежуток
    def __init__(self, hours, minutes): # Конструктор
        self.__hours = hours     # часы
        self.__minutes = minutes # минуты

    # Возвращает время в минутах
    def get_time_in_minutes(self):
        return self.__hours * 60 + self.__minutes

    # Возвращаем строку со значениями полей
    def get_str(self):
        return f'{self.__hours:>02}:{self.__minutes:>02}'

    def __eq__(self, other): # Перегрузка операции ==
        return (self.get_time_in_minutes() ==
                other.get_time_in_minutes())

    def __ne__(self, other): # Перегрузка операции !=
        return not (self == other)

    # Перегрузка операции <
    def __lt__(self, other):
        return (self.get_time_in_minutes() <
                other.get_time_in_minutes())
```

Вспомогательный метод, облегчает сравнение двух объектов класса Time

При перегрузке операций сравнения можно пользоваться уже имеющимися перегрузками. `not (self == other)` интерпретатор преобразует в `not (__eq__(self, other))`

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример перегрузки операций сравнения (2)

```
if __name__ == '__main__':
    # Создаём для проверки два экземпляра
    f = Time(2, 30) # 2 часа 30 минут
    s = Time(4, 0)  # 4 часа 0 минут

    if f == s: # Вызывается метод __eq__()
        print('Временные промежутки равны')
    elif f != s: # Вызывается метод __ne__()
        print(f.get_str(), 'не равно', s.get_str())

    # Создаём список из 4 временных промежутков
    ls = [f, s, Time(1, 10), Time(8, 35)]

    print(', '.join([i.get_str() for i in ls]))

    # Функция min() использует метод __lt__()
    print('Минимальное время:', min(ls).get_str())
```

Формируем список строк, который тут же объединяем через запятую в одну строку

```
Ответ:
02:30 не равно 04:00
02:30, 04:00, 01:10, 08:35
Минимальное время: 01:10
```

Для красивого вывода нужно вызывать метод get_str()

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Перегрузка арифметических операций

Простые арифметические операции используют **два операнда** (не изменяя их) **для получения третьей переменной**, которая возвращается в качестве результата. Для перегрузки операций в случаях, когда **левым операндом является объект вашего класса**, используются следующие методы:

Специальный метод	Операция	Значение
`__add__(self, other)`	`x + y`	Возвращает результат сложения x и y
`__sub__(self, other)`	`x - y`	Возвращает результат вычитания из x значения y
`__mul__(self, other)`	`x * y`	Возвращает результат умножения x на y
`__truediv__(self, other)`	`x / y`	Возвращает результат деления x на y
`__floordiv__(self, other)`	`x // y`	Возвращает результат деления x на y округлённый до наименьшего целого
`__mod__(self, other)`	`x % y`	Возвращает остаток от целочисленного деления x на y
`__pow__(self, other)`	`x ** y`	Возвращает результат возведения x в степень y

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример перегрузки арифметических операций (1)

```
class Time:
    '''Класс моделирует временной промежуток'''

    def __init__(self, hours = 0, minutes = 0):
        # Если в качестве параметра будут переданы только минуты,
        # то напрямую установить их нельзя. Для этого сначала
        # получаем только минуты, потом из них выделяем часы
        minutes += hours * 60
        self.__hours = int(minutes // 60)
        self.__minutes = int(minutes % 60)

    def get_time_in_minutes(self):
        return self.__hours * 60 + self.__minutes

    # Метод возвращает форматированную строку со значениями полей
    def get_str(self):
        # После двоеточия идут опции форматирования:
        # > выравнивает по правому краю
        # 0 добавляет ноль в начало, если есть только одна цифра
        # 2 ширина вывода значения
        return f'{self.__hours:>02}:{self.__minutes:>02}'
```

Вспомогательный метод, облегчает сложение и умножение объектов Time

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример перегрузки арифметических операций (2)

```
# Продолжение класса

# Перегрузка операции +. Реализуем её для трёх вариантов:
# 1) Time + Time. 2) Time + int. 3) Time + float
def __add__(self, other):
    # Проверяем тип параметра other
    # если other - объект класса Time
    if isinstance(other, Time):
        m = (self.get_time_in_minutes() +
             other.get_time_in_minutes())
        return Time(minutes=m) # Возвращаем новый объект Time

    # если other – целое или вещественное число
    elif isinstance(other, (int, float)):
        m = self.get_time_in_minutes() + other
        return Time(minutes=m)  # Возвращаем новый объект Time

    else:  # если other какого-то другого типа, то говорим,
        return NotImplemented  # что нет реализации
```

`isinstance` – проверяет принадлежность заданному типу

Даже если в m значение окажется больше 60, то в конструкторе всё нормально сократится

Сообщает интерпретатору о том, что перегрузка этой операции не реализована для других типов данных

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример перегрузки арифметических операций (2)

```
    # Продолжение класса
    # Перегрузка операции *. Реализуем для other - числа
    def __mul__(self, other):
        # Проверяем тип параметра other
        if isinstance(other, (int, float)):
            m = self.get_time_in_minutes() * other
            return Time(minutes=m) # Возвращаем новый объект Time
        else:  # если other какого-то другого типа, то говорим,
            return NotImplemented # что нет реализации

if __name__ == '__main__':
    lec = Time(1, 30) # 1 час 30 минут
    br = Time(0, 10)  # 0 часов 10 минут
    sm = lec + br     # Вызывается метод __add__()
    print('Лекция с переменной занимают', sm.get_str(), 'времени')
    print('3 лекции длятся', (lec * 3).get_str()) # вызов __mul__()
    lec + []
```

Сложение со списком не реализовано

```
Ответ:
Лекция с переменной занимают 01:40 времени
3 лекции длятся 04:30
TypeError: unsupported operand type(s) for
+: 'Time' and 'list'
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Перегрузка арифметических операций

для правого операнда

При выполнении арифметической операции подходящий метод сначала ищется у левого операнда. Если он там не будет найден (**объект вашего класса стоит справа**), то подходящий метод будет искаться в правом операнде. При этом используются следующие методы:

Специальный метод	Операция	Значение
`__radd__(self, other)`	`x + y`	Возвращает результат сложения x и y
`__rsub__(self, other)`	`x - y`	Возвращает результат вычитания из x значения y
`__rmul__(self, other)`	`x * y`	Возвращает результат умножения x на y
`__rtruediv__(self, other)`	`x / y`	Возвращает результат деления x на y
`__rfloordiv__(self, other)`	`x // y`	Возвращает результат деления x на y округлённый до наименьшего целого
`__rmod__(self, other)`	`x % y`	Возвращает остаток от целочисленного деления x на y
`__rpow__(self, other)`	`x ** y`	Возвращает результат возведения x в степень y

Можно перегрузить через методы со слайда 6 – в теле метода с префиксом r вызываете метод без префикса с поменянными местами параметрами.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример перегрузки арифметических операций (1)

для правого операнда

```
class Time: # Класс моделирует временной промежуток
    def __init__(self, hours = 0, minutes = 0):
        minutes += hours * 60
        self.__hours = int(minutes // 60)
        self.__minutes = int(minutes % 60)

    def get_time_in_minutes(self):
        return self.__hours * 60 + self.__minutes

    def get_str(self):
        return f'{self.__hours:>02}:{self.__minutes:>02}'

    # Перегрузка операции *. Реализуем для other - числа
    def __mul__(self, other):
        if isinstance(other, (int, float)):
            m = self.get_time_in_minutes() * other
            return Time(minutes=m) # Возвращаем новый объект Time
        else:  # если other какого-то другого типа, то говорим,
            return NotImplemented # что нет реализации

    # Перегрузка операции * в случае, когда Time правый операнд
    def __rmul__(self, other):
        return self.__mul__(other)
```

Вместо полной перегрузки операции (дублирования кода) вызываем уже реализованную перегрузку. Так можно делать, если порядок операндов не имеет значения.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример перегрузки арифметических операций (2)

для правого операнда

```
# Продолжение класса

# Перегрузка операции -. Реализуем для other - числа
def __sub__(self, other):
    if isinstance(other, (int, float)):
        m = self.get_time_in_minutes() - other
        return Time(minutes=m) # Возвращаем новый объект Time

    else:  # если other какого-то другого типа, то говорим,
        return NotImplemented # что нет реализации

# Перегрузка операции - в случае, когда Time правый операнд
# Поскольку для вычитания порядок следования операндов важен,
# то реализуем полностью
def __rsub__(self, other):
    if isinstance(other, (int, float)):
        m = other - self.get_time_in_minutes()
        return Time(minutes=m) # Возвращаем новый объект Time

    else:  # если other какого-то другого типа, то говорим,
        return NotImplemented # что нет реализации
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример перегрузки арифметических операций (3)

для правого операнда

```
if __name__ == '__main__':
    # Для проверки создаём два объекта
    lec = Time(1, 30) # 1 час 30 минут
    br = Time(0, 10)  # 0 часов 10 минут

    # Вызываем метод __mul__()
    print('2 лекции длятся', (lec * 2).get_str())
    # Вызываем метод __rmul__()
    sm = 4 * br
    print('4 перемены длятся', sm.get_str())

    # Вызываем метод __sub__()
    print('лекция - 30 минут =', (lec - 30).get_str())
    # Вызываем метод __rsub__()
    print('30 минут - перемена = ', (30 - br).get_str())
```

Вызывается метод __rmul__(self, other). 4 помещается в other.

```
Ответ:
2 лекции длятся 03:00
4 перемены длятся 00:40
лекция - 30 минут = 01:00
30 минут - перемена =  00:20
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Перегрузка арифметических операций

с изменением левого операнда

При выполнении составной операции присваивания происходит **изменение значения левого операнда с помощью значения правого операнда**. Результатом операции является левый операнд.

Специальный метод	Операция	Значение
`__iadd__(self, other)`	`x += y`	К x прибавляется y
`__isub__(self, other)`	`x -= y`	Из x вычитается y
`__imul__(self, other)`	`x *= y`	x умножается на y
`__itruediv__(self, other)`	`x /= y`	x делится на y
`__ifloordiv__(self, other)`	`x //= y`	x делится на y, после чего дополнительно округляется до наименьшего целого
`__imod__(self, other)`	`x %= y`	В x записывается остаток от целочисленного деления x на y
`__ipow__(self, other)`	`x **= y`	В x помещается результат возведения x в степень y

Примечание. Значения операций указаны здесь как пример использования операций над числами. Конкретные действия, которые будут выполняться этими операциями над объектами вашего класса, зависят от реализации этих методов.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример перегрузки арифметических операций (1)

с изменением левого операнда

```
class Time: # Класс моделирует временной промежуток
    def __init__(self, hours = 0, minutes = 0):
        # Добавляем поля внутри метода set_time_from_minutes.
        # В нём же проводим проверку правильности ввода
        self.set_time_from_minutes(hours * 60 + minutes)

    # Устанавливает время из целого числа, обозначающего минуты.
    # Добавлено т.к. планируется изменять сам экземпляр, а не
    # создавать новый (см. метод __isub__())
    def set_time_from_minutes(self, minutes):
        self.__hours = int(minutes // 60)
        self.__minutes = int(minutes % 60)

    def get_time_in_minutes(self):
        return self.__hours * 60 + self.__minutes

    def get_str(self):
        return f'{self.__hours:>02}:{self.__minutes:>02}'
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример перегрузки арифметических операций (2)

с изменением левого операнда

```
# Продолжение класса

def __add__(self, other): # Перегрузка операции +.
    if isinstance(other, Time):  # если other – экземпляр Time
        m = (self.get_time_in_minutes() +
             other.get_time_in_minutes())
        return Time(minutes=m) # Возвращаем новый объект Time

    elif isinstance(other, (int, float)):  # если other - число
        m = self.get_time_in_minutes() + other
        return Time(minutes=m) # Возвращаем новый объект Time

    else: # если other какого-то другого типа, то говорим,
        return NotImplemented # что нет реализации

# Перегрузка операции +=
def __iadd__(self, other): # Можно реализовать через операцию +
    self = self.__add__(other) # Можно так: self = self + other
    return self
```

Фактически, здесь создаётся новый экземпляр класса Time, которым мы заменяем self.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример перегрузки арифметических операций (3)

с изменением левого операнда

```
# Продолжение класса

def __isub__(self, other): # Перегрузка операции -=
    # Можно прописать реализацию полностью
    if isinstance(other, Time): # если other – экземпляр Time
        m = (self.get_time_in_minutes() -
             other.get_time_in_minutes())
        self.set_time_from_minutes(m)
        return self # Возвращаем изменённый self

    elif isinstance(other, (int, float)): # если other - число
        m = self.get_time_in_minutes() - other
        self.set_time_from_minutes(m)
        return self # Возвращаем изменённый self

    else: # если other какого-то другого типа, то говорим,
        return NotImplemented # что нет реализации
```

В __isub__() мы устанавливаем новые значения сразу в поля экземпляра self. Потом экземпляр self возвращаем.
Если бы писали __sub__(), то там self был бы неизменен. Вместо этого создавался и возвращался новый экземпляр.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример перегрузки арифметических операций (4)

с изменением левого операнда

```
if __name__ == '__main__':
    lec = Time(1, 30) # 1 час 30 минут
    br = Time(0, 10)  # 0 часов 10 минут

    print('Длительность лекции до =',
          lec.get_str(), 'id =', id(lec))
    lec -= br
    print('Длительность лекции после =',
          lec.get_str(), 'id =', id(lec))

    print('Длительность перемены до =',
          br.get_str(), 'id =', id(br))
    br += 10
    print('Длительность перемены после =',
          br.get_str(), 'id =', id(br))
```

Вызывается метод __isub__(), внутри которого мы меняем значения полей экземпляра lec. Из-за этого его **id не меняется**.

Вызывается метод __iadd__(), реализованный через метод __add__(). Внутри создаётся новый экземпляр, которым мы заменяем экземпляр br. Из-за этого его **id меняется**.

```
Ответ:
Длительность лекции до = 01:30 id = 53607920
Длительность лекции после = 01:20 id = 53607920
Длительность перемены до = 00:10 id = 54165168
Длительность перемены после = 00:20 id = 54164928
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Перегрузка методов для приведения к типам

Помимо операций сравнения и арифметических операций, в Python предусмотрена перегрузка ряда функций, используемых для приведения к встроенным типам и т.д. Некоторые из них представлены ниже:

Специальный метод	Исполь-зование	Значение
`__str__(self)`	`str(x)` `print(x)`	Возвращает строковое представление объекта x
`__repr__(self)`	`>>> x`	Возвращает «сырую» строку, использующуюся для внутреннего представления в Python (по ней можно воссоздать объект).
`__int__(self)`	`int(x)`	Возвращает целое число, полученное из x
`__float__(self)`	`float(x)`	Возвращает вещественное число, полученное из x
`__bool__(self)`	`if x` `while x`	Вызывается при проверке истинности (возвращает True/False). Если этот метод не определён, вызывается метод __len__() (объекты, имеющие ненулевую длину, считаются истинными).

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример перегрузки методов приведения к типам (1)

```
class Time: # Класс моделирует временной промежуток
    def __init__(self, hours = 0, minutes = 0):
        # Добавляем поля внутри метода set_time_from_minutes.
        self.set_time_from_minutes(hours * 60 + minutes)

    # Устанавливает время из целого числа, обозначающего минуты
    def set_time_from_minutes(self, minutes):
        self.__hours = int(minutes // 60)
        self.__minutes = int(minutes % 60)


    # Перегрузка используется внутри функций print() и str()
    def __str__(self):
        return f'{self.__hours:>02}:{self.__minutes:>02}'

    # Перегрузка используется при выводе в консоли
    def __repr__(self):
        # Строка должна быть такой, чтобы её выполнение привело к
        # созданию текущего объекта
        return f'Time({self.__hours}, {self.__minutes})'
```

Преобразовали метод get_str в перегрузку метода __str__. Синтаксис использования стал проще. См. стр. 7.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример перегрузки методов приведения к типам (2)

```
# Продолжение класса
```

Преобразовали метод get_time_in_minutes в перегрузку метода __int__. Синтаксис использования стал проще, а диапазон использования шире.

```
# Перегрузка используется при приведении к типу int
def __int__(self):  # Возвращает время в минутах
    return self.__hours * 60 + self.__minutes

def __eq__(self, other): # Перегрузка операции ==
    return int(self) == int(other)

def __lt__(self, other): # Перегрузка операции <
    return int(self) < int(other)

def __add__(self, other): # Перегрузка операции +
    # Проверяем тип параметра other
    if isinstance(other, (int, float, Time)):
        m = int(self) + int(other)
        return Time(minutes=m) # Возвращаем новый объект Time

    else: # если other какого-то другого типа, то говорим,
        return NotImplemented # что нет реализации
```

Для вычислений везде используем метод __int__()

Убрали лишнюю ветку elif

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример перегрузки методов приведения к типам (3)

```
if __name__ == '__main__':
    lec = Time(1, 30)
    br = Time(0, 10)

    print('Длительность лекции', lec, 'или', int(lec), 'мин')

    if not (lec == br):
        print(lec, 'не равно', br)

    print('лекция + перемена =', lec + br)
    print('лекция + 30 минут =', lec + 30)

    ls = [lec, br, Time(1, 10)]
    print(ls)

    print('Минимальное время:', min(ls))
```

Вызывается __str__()

Вызывается __int__()

Вызывается __eq__()

Вызывается __add__()

Вызывается __repr__()

Вызывается __lt__()

```
Ответ:
Длительность лекции 01:30 или 90 мин
01:30 не равно 00:10
лекция + перемена = 01:40
лекция + 30 минут = 02:00
[Time(1, 30), Time(0, 10), Time(1, 10)]
Минимальное время: 00:10
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Перегрузка методов для работы с последовательностями (1)

Для того, чтобы ваш класс можно было использовать как последовательность (например, обращаться напрямую с элементами из внутреннего списка), нужно реализовать перегрузку специальных методов.

При этом в зависимости от того, изменяемая у вас последовательность или неизменяемая, нужно перегружать разное количество методов.

Специальный метод	Использование	Значение
`__getitem__(self, key)`	`x[key]`	Возвращает элемент из последовательности с индексом key. Нужен для изменяемых и неизменяемых.
`__setitem__(self, key, value)`	`x[key]=value`	Присваивает элементу из последовательности с индексом key значение value. Нужен для изменяемых.
`__delitem__(self, key)`	`del x[key]`	Определяет поведение при удалении элемента с индексом key. Нужен для изменяемых.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Перегрузка методов для работы с последовательностями (2)

Специальный метод	Исполь-зование	Значение
`__len__(self)`	`len(x)`	Возвращает количество элементов в последовательности. Нужен для изменяемых и неизменяемых.
`__iter__(self)`	`for i in x`	Возвращает итератор для последовательности (специальный объект, который хранит текущий элемент и «знает» где располагается следующий элемент). Итераторы возвращаются для встроенной функции iter() и в случае перебора элементов последовательности. Нужен для изменяемых и неизменяемых.
`__contains__(self, item)`	`item in x`	Предназначен для проверки принадлежности элемента с помощью in и not in. Нужен для изменяемых и неизменяемых.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример перегрузки методов для работы с последовательностями (1)

```
class TV: # Класс моделирует телевизор
    # Значения по умолчанию позволяют создавать без параметров
    def __init__(self, name = '', brand = '', diag = 0.0, price = 0.0):
        self.__name = name
        self.__brand = brand
        self.__diag = diag
        self.__price = price

    # Метод заполняет поля с клавиатуры
    def input_from_keyboard(self):
        self.__name = input('Введите название телевизора: ')
        self.__brand = input('Введите название фирмы: ')
        self.__diag = float(input('Введите диагональ телевизора: '))
        self.__price = float(input('Введите цену телевизора: '))
        return self # Возвращаем в качестве результата заполненный

    # Перегрузка метода приведения к строковому типу
    def __str__(self):
        return (f'Телевизор {self.__name} компании {self.__brand} '
                f'диагональю {self.__diag} и '
                f'стоимостью {self.__price} руб.')

    # Проверяем, совпадает ли значение в поле __brand
    def is_this_brand(self, brand): # с переданным параметром
        return self.__brand.lower() == brand.lower()
    # Проверяем, находится ли цена в заданном диапазоне
    def is_in_this_price_range(self, low, high):
        return low <= self.__price <= high
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример перегрузки методов для работы с последовательностями (2)

```
class TV_list: # Класс моделирует список телевизор
    # Для инициализации можно передать список экземпляров TV
    def __init__(self, tvs = []):
        self.__tvs = tvs
    def input_from_keyboard(self): # Заполнение списка телевизоров
        num = int(input('Сколько телевизоров вводите? '))
        for _ in range(num):
            tv = TV().input_from_keyboard()
            self.__tvs.append(tv)

    # Специальные методы для работы с TV_list как со списком

    # Возвращает телевизор по индексу key
    def __getitem__(self, key):
        return self.__tvs[key]
    # Устанавливает новый телевизор value по индексу key
    def __setitem__(self, key, value):
        self.__tvs[key] = value
    # Удаляет телевизор по индексу key
    def __delitem__(self, key):
        del self.__tvs[key]

    # Позволяет перебирать список телевизоров в цикле
    def __iter__(self):
        return iter(self.__tvs)
    # Проверяет наличие телевизора item в списке
    def __contains__(self, item):
        return item in self.__tvs
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример перегрузки методов для работы с последовательностями (3)

```
# Дополнительные методы
# Перегрузка метода возвращает количество телевизоров
def __len__(self):
    return len(self.__tvs)
# Позволяет добавлять телевизор в конец списка
def append(self, tv):
    self.__tvs.append(tv)

# Перегрузка метода приведения к строковому типу
def __str__(self):
    st = 'Телевизоры:\n'
    st += '\n'.join([str(tv) for tv in self])
    return st

# Метод возвращает новый экземпляр класса TV_list, включающий
# только те телевизоры, которые удовлетворяют условию
def get_tvs_of_this_brand(self, brand):
    # Создаём список телевизоров путём выбора необходимых
    result = [tv for tv in self
              if tv.is_this_brand(brand)]
    # Преобразуем его в экземпляр класса TV_list
    return TV_list(result)

def get_tvs_in_this_price_range(self, low, high):
    result = [tv for tv in self
              if tv.is_in_this_price_range(low, high)]
    return TV_list(result)
```

В цикле для self автоматически вызывается метод __iter__(), который последовательно возвращает телевизоры из списка self.__tvs

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример перегрузки методов для работы с последовательностями (4)

```
# Основная программа
tvs = TV_list() # Создаём объект класса TV_list
tvs.input_from_keyboard() # Заполняем список телевизоров

# Выводим все телевизоры на экран
print('\n', tvs)

# Обращаемся с объектом tvs как со списком
print('\nПервый телевизор: ', tvs[0])

# Добавляем новый телевизор
tv = TV('LG1256', 'LG', 56, 58000)
tvs.append(tv)

print('\n', tvs)

min_price = float(input('Какая минимальная цена? '))
max_price = float(input('Какая максимальная цена? '))
# Выбираем все телевизоры, удовлетворяющие условию вхождения в диапазон
tv_price = tvs.get_tvs_in_this_price_range(min_price, max_price)

# Выводим их на экран
print(tv_price)
```

Вызывается __str__()

Вызывается __getitem__()

```
Сколько телевизоров вводите? 2
Введите название телевизора: Philips 64598
Введите название фирмы: Philips
Введите диагональ телевизора: 54
Введите цену телевизора: 1354989
Введите название телевизора: ASUS 98465
Введите название фирмы: ASUS
Введите диагональ телевизора: 89
Введите цену телевизора: 5684896
 Телевизоры:
Телевизор Philips 64598 компании Philips диагональю 54.0 и
стоимостью 1354989.0 руб.
Телевизор ASUS 98465 компании ASUS диагональю 89.0 и стоимостью
5684896.0 руб.

Первый телевизор:  Телевизор Philips 64598 компании Philips
диагональю 54.0 и стоимостью 1354989.0 руб.

 Телевизоры:
Телевизор Philips 64598 компании Philips диагональю 54.0 и
стоимостью 1354989.0 руб.
Телевизор ASUS 98465 компании ASUS диагональю 89.0 и стоимостью
5684896.0 руб.
Телевизор LG1256 компании LG диагональю 56 и стоимостью 58000 руб.

Какую фирму ищете? ASUS
Телевизоры:
Телевизор ASUS 98465 компании ASUS диагональю 89.0 и стоимостью
5684896.0 руб.
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева